

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

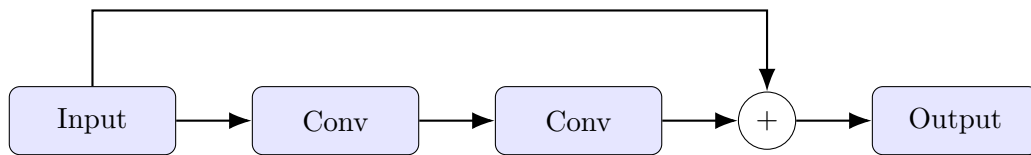


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

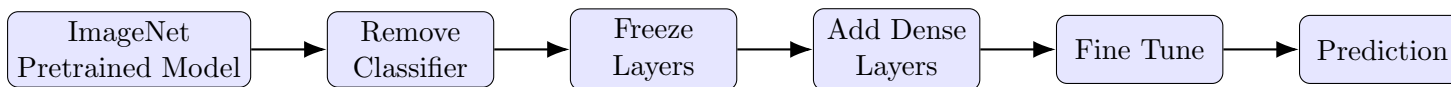


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.

4. Print the dimensions of the training and testing datasets.



Figure 3: Ten sample images from the CIFAR-10 dataset

Inference:

Training Set Shape	50000
Testing Set Shape	10000

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Model chosen: VGG16

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10-20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

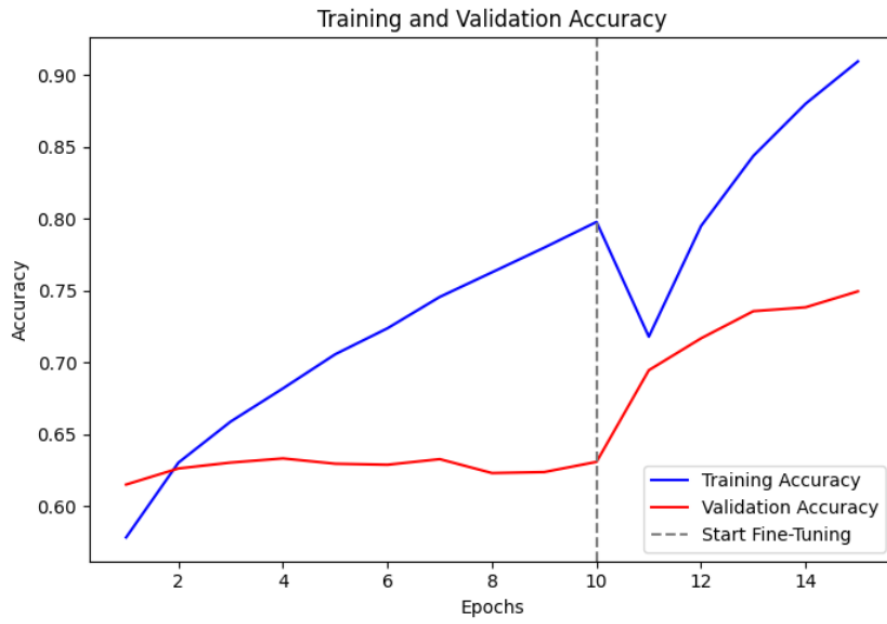


Figure 4: Training Accuracy vs Validation Accuracy over Epochs

Inference:

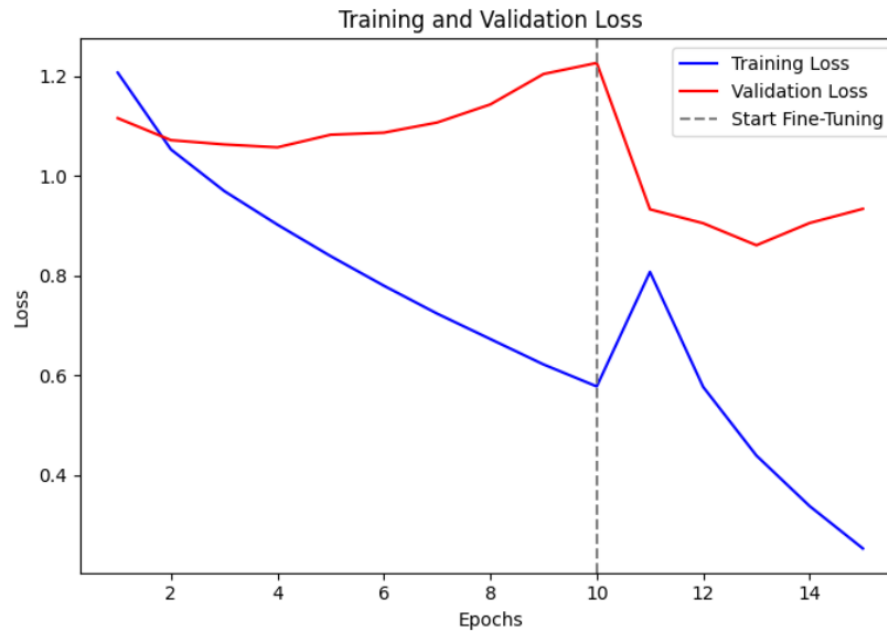


Figure 5: Training Loss vs Validation Loss over Epochs

Inference:

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Accuracy Before Fine Tuning	63.07%
Accuracy After Fine Tuning	74.94%

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Accuracy	73.70%
Precision	0.7476
Recall	0.7370
F1-score	0.7375

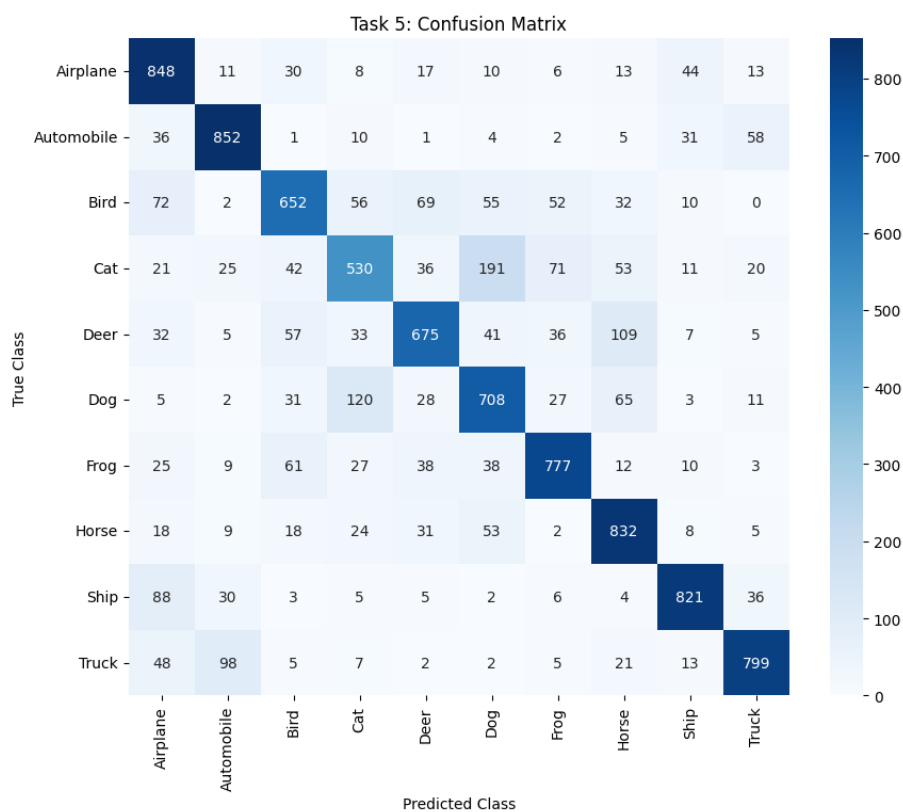


Figure 6: Confusion Matrix for the Test Set Predictions

Inference:

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

Combo	Learning Rate	Batch Size	Optimizer	Dense Units	Freeze	Val Accuracy (%)
1	0.001	16	Adam	128	True	62.47
2	0.001	16	Adam	128	False	77.71
3	0.001	16	Adam	256	True	63.45
4	0.001	16	Adam	256	False	77.72
5	0.001	16	SGD	128	True	61.40
6	0.001	16	SGD	128	False	76.57
7	0.001	16	SGD	256	True	62.09
8	0.001	16	SGD	256	False	77.52
9	0.001	32	Adam	128	True	62.88
10	0.001	32	Adam	128	False	77.95
11	0.001	32	Adam	256	True	63.26
12	0.001	32	Adam	256	False	77.79
13	0.001	32	SGD	128	True	60.48
14	0.001	32	SGD	128	False	76.79
15	0.001	32	SGD	256	True	60.69
16	0.001	32	SGD	256	False	78.29
17	0.001	64	Adam	128	True	63.34
18	0.001	64	Adam	128	False	79.10
19	0.001	64	Adam	256	True	63.39
20	0.001	64	Adam	256	False	77.03
21	0.001	64	SGD	128	True	59.83
22	0.001	64	SGD	128	False	75.04
23	0.001	64	SGD	256	True	59.80
24	0.001	64	SGD	256	False	77.05
25	0.0001	16	Adam	128	True	61.16
26	0.0001	16	Adam	128	False	78.58
27	0.0001	16	Adam	256	True	61.88
28	0.0001	16	Adam	256	False	78.67
29	0.0001	16	SGD	128	True	56.37
30	0.0001	16	SGD	128	False	74.13
31	0.0001	16	SGD	256	True	56.33
32	0.0001	16	SGD	256	False	74.66

Combo	Learning Rate	Batch Size	Optimizer	Dense Units	Freeze	Val Accuracy (%)
33	0.0001	32	Adam	128	True	60.99
34	0.0001	32	Adam	128	False	79.17
35	0.0001	32	Adam	256	True	61.67
36	0.0001	32	Adam	256	False	78.57
37	0.0001	32	SGD	128	True	52.57
38	0.0001	32	SGD	128	False	71.08
39	0.0001	32	SGD	256	True	52.49
40	0.0001	32	SGD	256	False	71.84
41	0.0001	64	Adam	128	True	60.14
42	0.0001	64	Adam	128	False	79.62
43	0.0001	64	Adam	256	True	61.10
44	0.0001	64	Adam	256	False	79.42
45	0.0001	64	SGD	128	True	47.84
46	0.0001	64	SGD	128	False	67.42
47	0.0001	64	SGD	256	True	48.45
48	0.0001	64	SGD	256	False	68.13

17. Results

17.1 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	5,407,286	59.94	157.74 s
AlexNet	57,044,810	57.28	206.67 s
VGG16	134,301,514	69.23	2310.55 s
GoogleNet	5,610,154	72.03	654.23 s
ResNet-18	11,181,642	74.97	523.34 s

18. Discussion Questions

Answer the following questions.

1. Why is AlexNet considered a breakthrough in deep learning?
2. Why does VGG16 use only 3×3 convolution filters?
3. Explain the advantages of the Inception module.
4. What is the purpose of residual learning?
5. Differentiate LeNet and ResNet.
6. What is Transfer Learning?
7. Why is fine tuning required?
8. Explain the difference between dilated convolution and transpose convolution.
9. Why do pretrained models converge faster?
10. Compare the computational complexity of LeNet and ResNet.

Answers

1. AlexNet is considered a breakthrough because it was the first deep CNN to win the ImageNet Large Scale Visual Recognition Challenge by a large margin. It introduced the use of ReLU

activation for faster convergence, Dropout to reduce overfitting, and efficient GPU-based training, proving that deep networks could be trained successfully on large-scale image datasets.

2. VGG16 uses only 3×3 convolution filters because stacking multiple small filters gives the same effective receptive field as a larger filter (e.g., two 3×3 layers approximate a 5×5 filter) while using fewer parameters and adding more non-linearity through additional activation layers, which improves feature learning.
3. The Inception module allows the network to extract features at multiple scales simultaneously by applying 1×1 , 3×3 and 5×5 convolutions in parallel along with pooling, and concatenating the results. The 1×1 convolutions before larger filters reduce dimensionality and computational cost, making the network both accurate and efficient.
4. The purpose of residual learning is to make it easier for very deep networks to be optimized. Instead of learning a direct mapping $H(x)$, the network learns a residual function $F(x) = H(x) - x$, allowing the identity mapping to be learned easily when needed. This mitigates the vanishing gradient problem and allows gradients to flow directly through skip connections.
5. LeNet is a shallow network (7 layers) designed for small grayscale digit images with a small number of parameters, whereas ResNet is a much deeper network (up to 50+ layers) that uses residual/skip connections to enable stable training of very deep architectures on large, complex, colour image datasets.
6. Transfer Learning is a technique in which a model trained on one large dataset (such as ImageNet) is reused, either directly or after fine tuning, as the starting point for a model on a new, often smaller, related task, instead of training a new model from scratch.
7. Fine tuning is required because the features learned by a pretrained model on a general dataset may not be perfectly suited to a new, specific dataset. Unfreezing and retraining some of the later convolutional layers allows the model to adapt its learned features to the new task, improving accuracy.
8. Dilated convolution increases the receptive field of a filter by inserting gaps (dilation) between kernel elements, without adding extra parameters or reducing spatial resolution, and is mainly used for tasks like segmentation. Transpose convolution, in contrast, performs learnable upsampling to increase the spatial dimensions of a feature map and is mainly used for generating or reconstructing images, such as in autoencoders and GANs.
9. Pretrained models converge faster because their convolutional layers have already learned general low-level and mid-level features (edges, textures, shapes) from a large dataset. The new model only needs to learn how to combine these existing features for the new task, rather than learning all features from random initialization.
10. LeNet has very low computational complexity due to its small size (about 60K parameters) and shallow depth, making it fast but limited in representational power. ResNet, despite being much deeper (50+ layers, around 25.6M parameters for ResNet50), remains computationally efficient relative to its depth because of bottleneck residual blocks, but it still requires significantly more computation than LeNet due to its greater depth and parameter count.

19. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

20. Source Code

1. Implement transfer learning using VGG16.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>